กลุ่ม 3 (Group 3) | รายวิชา Bioinformatics Programming Project II (20%) | กำหนดส่ง 17 พฤศจิกายน 2569
สมาชิก: **[1]** ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1) · **[2]** ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2) · **[3]** ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3) · **[4]** ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4) · **[5]** ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

โครงสร้างฟังก์ชัน ลำดับการทำงาน (Flowchart) และรายละเอียด Input / Action / Output

1. แผนผังลำดับการทำงานของโปรแกรม (System Execution Flowchart: Step 0 → Step 5)

โปรแกรมทำงานแบบเป็นลำดับขั้น (Sequential & Modular Pipeline) โดยมีฟังก์ชันหลัก `run_amr_prediction_pipeline()` ในโมดูล `main_pipeline.py` เป็นตัวควบคุมกลาง และมีการเรียกใช้ฟังก์ชันข้ามโมดูลย่อย (เช่น โมดูลที่ 3 เรียกใช้ `parse_fasta_records()` จากโมดูลที่ 2 และโมดูลที่ 5 ใช้ตารางเกณฑ์ CLSI จากโมดูลที่ 4):

[ลำดับที่ 0] START · Main Module
ไฟล์: `main_pipeline.py`
ฟังก์ชัน: `run_amr_prediction_pipeline()`
รับพารามิเตอร์จากผู้เรียกหรือ CLI (`argparse`) และควบคุมการส่งข้อมูลผ่าน Step 1 ➔ Step 5

[ลำดับที่ 1] Module 1 · Clinical Phenome
ไฟล์: `mod1_clinical_phenome.py`
ฟังก์ชัน: `load_and_profile_clinical_resistance()`
อ่าน CSV 574,239 แถว คำนวณสัดส่วน S/NS, MIC_{50}/MIC_{90} และอัตราเชื้อ MDR

[ลำดับที่ 2] Module 2 · Genome Fetcher
ไฟล์: `mod2_genome_fetcher.py`
ฟังก์ชัน: `fetch_and_parse_isolate_genomes()`
โหลด FASTA 20 สายพันธุ์ + JSON จาก NCBI/ENA และตรวจคุณภาพ %GC, %N

[ลำดับที่ 3] Module 3 · AMR Gene Scanner
ไฟล์: `mod3_amr_gene_scanner.py`
ฟังก์ชัน: `scan_amr_genes_and_mutations()`
สแกนยีนดื้อยา 7 ยีนด้วย K-mer และแปลรหัสตรวจ SNPs ใน `gyrA`, `parC`, `nfsA`, `nfsB`

[ลำดับที่ 4] Module 4 · Genotype-Drug Map
ไฟล์: `mod4_genotype_phenotype_map.py`
ฟังก์ชัน: `map_genetic_determinants_to_drugs()`
จับคู่ยีนและการกลายพันธุ์กับกลไกยา 5 ชนิด คำนวณคะแนนความเสี่ยงดื้อยาและตัวคูณ MIC

[ลำดับที่ 5] Module 5 · Predictor & END
ไฟล์: `mod5_resistance_predictor.py`
ฟังก์ชัน: `predict_susceptibility_and_evaluate()`
ทำนาย S/NS & MIC, ระบุสายพันธุ์ MDR, วัดค่าความแม่นยำ และบันทึกไฟล์ CSV/JSON

2. รายละเอียด ข้อมูลเข้า (Input), การดำเนินงาน (Action) และผลลัพธ์ (Output) ของแต่ละโมดูล

โมดูลและชื่อฟังก์ชัน	ข้อมูลเข้า (Input & Types)	การดำเนินงาน (Action / Algorithm)	ผลลัพธ์ (Output & Return Type)
Main Module (ฟังก์ชันหลัก) `main_pipeline.py` • `run_amr_prediction_pipeline()` • `main()`	• `clinical_csv_path` (`str|Path`) • `isolate_fasta_path` (`str|Path`) • `isolate_metadata_path` (`str|Path`) • `reference_amr_fasta_path` • `output_dir`, `fetch_online` (`bool`)	1. รับพารามิเตอร์จากผู้เรียกหรือ CLI (`argparse`) 2. เรียกใช้ฟังก์ชันย่อยใน Module 1 ➔ 2 ➔ 3 ➔ 4 ➔ 5 ตามลำดับ 3. ส่งผ่านโครงสร้างข้อมูลที่ได้จากโมดูลก่อนหน้าไปยังโมดูลถัดไป 4. แสดงตารางสรุปผลการรันทางหน้าจอและคืนค่าพจนานุกรมสรุปผล	`dict[str, Any]`: รายงานสรุปผลการวิเคราะห์คลินิก ผลการทำนายจีโนม ค่าสถิติความแม่นยำ และพาธไฟล์ผลลัพธ์
Module 1 (สมาชิกคนที่ 1) `mod1_clinical_phenome.py` • `load_and_profile_clinical_resistance()` • `calculate_mic_percentiles()` • `summarize_isolate_mdr_patterns()`	• `csv_path` (`str|Path`): ไฟล์ตารางคลินิก 574,239 แถว • `target_drugs` (`tuple[str,...]`): `('AMC','AMP','CIP','NIT','TET')`	1. อ่านไฟล์ CSV แบบ Streaming ด้วย `csv.DictReader` และตรวจสอบคอลัมน์ 2. แยกนับจำนวน S/NS และรวบรวมค่า MIC ของยาแต่ละชนิด 3. คำนวณค่า MIC_{50}, MIC_{90} และค่าเฉลี่ยเรขาคณิต (Geometric Mean MIC) 4. จัดกลุ่มสายพันธุ์ที่ดื้อยา ≥ 3 กลุ่มยาเพื่อคำนวณอัตราเชื้อ MDR	`dict[str, Any]` (`ClinicalResistanceProfile`): สถิติอัตราการดื้อยา ค่า MIC_{50}/MIC_{90} แยกตามตัวยา แหล่งติดเชื้อ และสรุปสัดส่วน MDR
Module 2 (สมาชิกคนที่ 2) `mod2_genome_fetcher.py` • `fetch_and_parse_isolate_genomes()` • `parse_fasta_records()` • `compute_sequence_qc_metrics()`	• `fasta_path` (`str|Path`): ลำดับจีโนมสายพันธุ์ *E. coli* • `metadata_path` (`str|Path`): ข้อมูล NCBI/ENA JSON • `fetch_online` (`bool`)	1. แยกส่วน Header และรวมลำดับเบสจากไฟล์ FASTA เป็นตัวพิมพ์ใหญ่ 2. ตรวจสอบอักขระเบสมาตรฐาน IUPAC และคำนวณความยาว, %GC, %N 3. คัดกรองสายพันธุ์ตามเกณฑ์ QC (ความยาว ≥ 300 bp, %N ≤ 5%, %GC 35–65%) 4. ผสานข้อมูลรหัส NCBI Assembly, ENA Accession และฟีโนไทป์อ้างอิง	`list[dict[str, Any]]` (`IsolateGenome`): รายการข้อมูลจีโนม 20 สายพันธุ์พร้อมลำดับเบสและผลตรวจคุณภาพ QC
Module 3 (สมาชิกคนที่ 3) `mod3_amr_gene_scanner.py` • `scan_amr_genes_and_mutations()` • `align_and_score_gene_hit()` • `translate_bacterial_cds()` • `detect_qrdr_and_nitroreductase_mutations()`	• `isolate_genomes` (`list[dict]`) จาก Module 2 • `reference_fasta_path` (`str|Path`): ยีนอ้างอิง 11 ยีน • `min_identity` (`0.90`) • `min_coverage` (`0.85`)	1. เรียกใช้ `parse_fasta_records()` จาก Module 2 เพื่อโหลดยีนอ้างอิง 2. ค้นหาตำแหน่งยีนด้วย K-mer Seed (k=15) และคำนวณ %Identity, %Coverage 3. แปลรหัสพันธุกรรม (Codon Table 11) ของยีน `gyrA`, `parC`, `nfsA`, `nfsB` 4. เทียบกรดอะมิโนกับตำแหน่งสำคัญ (`gyrA` 83/87, `parC` 80/84, `nfsA` 15, `nfsB` 44/94)	`dict[str, dict[str, Any]]`: รายการยีนดื้อยาที่พบ (Acquired AMR genes) และรหัสการกลายพันธุ์ระดับกรดอะมิโนของแต่ละสายพันธุ์
Module 4 (สมาชิกคนที่ 4) `mod4_genotype_phenotype_map.py` • `map_genetic_determinants_to_drugs()` • `evaluate_drug_mechanism_score()`	• `scan_results` (`dict`) จาก Module 3 • `clinical_profile` (`dict`) จาก Module 1	1. จับคู่ยีนและการกลายพันธุ์เข้ากับกฎกลไกชีวโมเลกุลของยาทั้ง 5 ชนิด 2. ประเมินผลเสริมฤทธิ์ (เช่น การกลายพันธุ์คู่ใน `gyrA+parC` ต่อยา `CIP` หรือการสูญเสียหน้าที่ของทั้ง `nfsA+nfsB` ต่อยา `NIT`) 3. คำนวณคะแนนกลไกพันธุกรรม (`genetic_score`) ถ่วงน้ำหนักร่วมกับ `baseline_ns_rate` จาก Module 1 และประเมินตัวคูณ MIC (`fold_increase`)	`dict[str, dict[str, Any]]`: คะแนนความเสี่ยงการดื้อยา คำอธิบายกลไกระดับโมเลกุล และตัวคูณ MIC ของทั้ง 5 ตัวยาต่อสายพันธุ์
Module 5 (สมาชิกคนที่ 5) `mod5_resistance_predictor.py` • `predict_susceptibility_and_evaluate()` • `classify_mic_and_phenotype()` • `compute_confusion_and_concordance()`	• `mapped_profiles` (`dict`) จาก Module 4 • `clinical_profile` (`dict`) จาก Module 1 • `isolate_genomes` (`list`) จาก Module 2 • `output_dir` (`str|Path`)	1. คำนวณค่า `predicted_mic` (`wildtype_mic` × `fold_increase`) และเทียบเกณฑ์ CLSI Breakpoint เพื่อจำแนกสถานะ S หรือ NS 2. นับจำนวนกลุ่มยาที่ทำนายเป็น NS เพื่อระบุสถานะสายพันธุ์ดื้อยาหลายขนาน (MDR) 3. สร้าง Confusion Matrix (TP, TN, FP, FN) คำนวณ Accuracy, Sensitivity, Specificity, F1 4. บันทึกไฟล์ผลลัพธ์ `amr_predictions.csv` และ `project2_evaluation_summary.json`	`dict[str, Any]` พร้อมไฟล์ตาราง `amr_predictions.csv` (100 รายการทำนาย) และไฟล์ JSON สรุปความแม่นยำ

กลุ่ม 3 (Group 3) | รายวิชา Bioinformatics Programming Project II (20%) | กำหนดส่ง 17 พฤศจิกายน 2569
สมาชิก: **[1]** ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1) · **[2]** ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2) · **[3]** ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3) · **[4]** ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4) · **[5]** ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

รหัสเทียม (Pseudocode) แสดงลำดับการทำงาน และผลลัพธ์จากการรันโปรแกรมจริง

3. รหัสเทียมแสดงลำดับขั้นตอนการทำงานของโปรแกรม (Structured Pseudocode)

```
ALGORITHM Ecoli_AMR_Genomic_Clinical_Prediction_Pipeline
INPUT:  clinical_csv_path, isolate_fasta_path, isolate_metadata_path, reference_amr_fasta_path, output_dir
OUTPUT: amr_predictions.csv, project2_evaluation_summary.json, evaluation_summary_dict
1. FUNCTION run_amr_prediction_pipeline(clinical_csv_path, isolate_fasta_path, isolate_metadata_path, reference_amr_fasta_path, output_dir):
     // [ลำดับที่ 1] เรียกใช้ Module 1 เพื่อสกัดโปรไฟล์ความเสี่ยงคลินิกและค่าสถิติ MIC จากข้อมูล 574,239 แถว
     clinical_profile <- CALL Module1.load_and_profile_clinical_resistance(clinical_csv_path)
       ├─ STREAM rows FROM clinical_csv_path WHERE drug_code IN ('AMC', 'AMP', 'CIP', 'NIT', 'TET')
       ├─ FOR EACH drug_code: COMPUTE S_rate, NS_rate, MIC50, MIC90, Geometric_Mean_MIC
       └─ FOR EACH isolate_ID: COUNT resistant drug_classes; IF count >= 3 THEN MARK AS Clinical_MDR
     // [ลำดับที่ 2] เรียกใช้ Module 2 เพื่ออ่านลำดับเบสจีโนม FASTA ของสายพันธุ์ E. coli และตรวจสอบคุณภาพลำดับเบส
     isolate_genomes <- CALL Module2.fetch_and_parse_isolate_genomes(isolate_fasta_path, isolate_metadata_path)
       ├─ fasta_records <- CALL Module2.parse_fasta_records(isolate_fasta_path)
       └─ FOR EACH record IN fasta_records:
            qc <- CALL Module2.compute_sequence_qc_metrics(record.sequence) // ตรวจความยาว, %GC, %N
            MERGE record WITH NCBI/ENA metadata AND qc INTO isolate_genomes
     // [ลำดับที่ 3] เรียกใช้ Module 3 เพื่อสแกนหายีนดื้อยาและตรวจหาการกลายพันธุ์ระดับกรดอะมิโน
     scan_results <- CALL Module3.scan_amr_genes_and_mutations(isolate_genomes, reference_amr_fasta_path)
       ├─ ref_genes <- CALL Module2.parse_fasta_records(reference_amr_fasta_path) // เรียกใช้ฟังก์ชันข้ามโมดูล
       └─ FOR EACH isolate IN isolate_genomes:
            FOR EACH acquired_gene IN ('blaTEM-1', 'blaCTX-M-15', 'blaOXA-1', 'ampC', 'qnrS1', 'tetA', 'tetB'):
              hit <- CALL Module3.align_and_score_gene_hit(isolate.sequence, ref_genes[acquired_gene], kmer=15)
              IF hit.identity >= 0.90 AND hit.coverage >= 0.85 THEN APPEND acquired_gene TO isolate.acquired_hits
            FOR EACH core_gene IN ('gyrA', 'parC', 'nfsA', 'nfsB'):
              aligned_cds <- CALL Module3.align_and_score_gene_hit(isolate.sequence, ref_genes[core_gene])
              aa_seq <- CALL Module3.translate_bacterial_cds(aligned_cds) // แปลรหัส Codon Table 11
              CHECK hotspot codons (gyrA 83,87; parC 80,84; nfsA 15; nfsB 44,94) AGAINST wild-type AA
     // [ลำดับที่ 4] เรียกใช้ Module 4 เพื่อแปลงยีนดื้อยาและการกลายพันธุ์เป็นคะแนนกลไกต่อยาปฏิชีวนะ 5 ชนิด
     mapped_profiles <- CALL Module4.map_genetic_determinants_to_drugs(scan_results, clinical_profile)
       └─ FOR EACH isolate_id, scan_info IN scan_results:
            FOR EACH drug_code IN ('AMC', 'AMP', 'CIP', 'NIT', 'TET'):
              score_info <- CALL Module4.evaluate_drug_mechanism_score(drug_code, scan_info.genes, scan_info.mutations, baseline_ns)
     // [ลำดับที่ 5] เรียกใช้ Module 5 เพื่อทำนายฟีโนไทป์ S/NS, คาดการณ์ค่า MIC, ระบุสายพันธุ์ MDR และประเมินผล
     evaluation_summary <- CALL Module5.predict_susceptibility_and_evaluate(mapped_profiles, clinical_profile, isolate_genomes, output_dir)
       ├─ FOR EACH isolate_id, drug_code:
       │    pred_mic <- wildtype_mic[drug_code] * score_info.estimated_fold_increase
       │    pred_phenotype <- 'NS' IF (pred_mic >= CLSI_ns_breakpoint[drug_code] OR prob_ns >= 0.50) ELSE 'S'
       ├─ IF count(NS_drug_classes of isolate) >= 3 THEN predicted_mdr <- True
       ├─ metrics <- CALL Module5.compute_confusion_and_concordance(predictions, actual_phenotypes)
       └─ WRITE amr_predictions.csv AND project2_evaluation_summary.json TO output_dir
     RETURN evaluation_summary
```

4. สรุปผลลัพธ์จากการรันโปรแกรมด้วยข้อมูลจริง (Actual Program Output Summary)

ก. ผลวิเคราะห์ฐานข้อมูลคลินิก (Module 1: 574,239 รายการ / 130,407 สายพันธุ์)

รหัสยา (กลุ่มยา)	ทดสอบ	ไว (S)	ดื้อ (NS)	$MIC_{50/90}$
AMP (Penicillins)	130,212	52.67%	**47.33%**	8 / 32
TET (Tetracyclines)	105,698	70.76%	**29.24%**	1 / 16
CIP (Quinolones)	130,255	78.30%	**21.70%**	0.25 / 4
AMC (Penicillin+Inh)	105,657	82.63%	**17.37%**	4 / 16
NIT (Nitrofurans)	102,417	94.64%	**5.36%**	16 / 32

- **ไวต่อยาทุกกลุ่ม (Pan-susceptible):** 59,144 สายพันธุ์ (45.35%)
- **ดื้อยาหลายขนาน (MDR ≥ 3 กลุ่มยา):** 13,817 สายพันธุ์ (**10.60%**)

ข. ผลการทำนายจากจีโนม NCBI/ENA และการประเมินความแม่นยำ (Module 2–5)

ตัวชี้วัดประสิทธิภาพ (100 คู่สายพันธุ์-ยา)	ผลการประเมิน
จำนวนจีโนมที่ผ่าน QC / จำนวนคู่ทดสอบทั้งหมด	20 สายพันธุ์ / 100 คู่
True Positive (NS) / True Negative (S)	43 / 57 (FP=0, FN=0)
ความแม่นยำจำแนกฟีโนไทป์ (Accuracy / F1)	**100.00% / 1.0000**
ความไว (Sensitivity) / ความจำเพาะ (Specificity)	**100.00% / 100.00%**
ความตรงกันของค่า MIC (Exact MIC Match)	**89.00% (89/100)**
จำนวนสายพันธุ์ที่ทำนายเป็น MDR ในกลุ่มจีโนม	8 / 20 (40.00%)

- **รันโปรแกรม:** `python3 project2/main_pipeline.py`
- **รันชุดทดสอบ:** `python3 -m unittest discover -s project2/tests -v`

5. ตัวอย่างผลลัพธ์การทำนายรายสายพันธุ์จากไฟล์ `amr_predictions.csv`

รหัสสายพันธุ์ (NCBI / ENA)	ยีนดื้อยาและการกลายพันธุ์ที่ตรวจพบ (Module 3)	ผลทำนายฟีโนไทป์และค่า MIC (µg/mL) ต่อยาทั้ง 5 ชนิด	สถานะ MDR
EC_GEN_001 (GCF_000005845.2 / U00096.3)	ไม่พบยีนดื้อยา (Wild-type K-12 Reference)	AMC: 4 (S) · AMP: 4 (S) · CIP: 0.25 (S) · NIT: 16 (S) · TET: 2 (S)	Non-MDR (0 กลุ่ม)
EC_GEN_003 (GCF_001683595.1 / KC844055.1)	blaTEM-1 (Identity 100.0%)	AMC: 8 (S) · **AMP: 32 (NS)** · CIP: 0.25 (S) · NIT: 16 (S) · TET: 2 (S)	Non-MDR (1 กลุ่ม)
EC_GEN_008 (GCF_000285655.3 / MF464374.1)	blaCTX-M-15, blaOXA-1, tetA + gyrA_S83L/D87N, parC_S80I	**AMC: 16 (NS)** · **AMP: 32 (NS)** · **CIP: 4 (NS)** · NIT: 16 (S) · **TET: 16 (NS)**	**MDR (3 กลุ่มยา)**
EC_GEN_009 (GCF_003345125.1 / PX680826.1)	nfsA_R15C (Missense) + nfsB_W94* (Nonsense)	AMC: 4 (S) · AMP: 4 (S) · CIP: 0.25 (S) · **NIT: 128 (NS)** · TET: 2 (S)	Non-MDR (1 กลุ่ม)
EC_GEN_020 (GCF_009759685.1 / CP046396.1)	blaCTX-M-15, blaOXA-1, ampC, tetB + gyrA_S83L/D87N, parC_S80I/E84V, nfsA_R15*, nfsB_Q44*	**AMC: 32 (NS)** · **AMP: 32 (NS)** · **CIP: 4 (NS)** · **NIT: 256 (NS)** · **TET: 16 (NS)**	**MDR/XDR (4 กลุ่ม)**